QuickTIMS / CalTrack — Developer Onboarding & Setup Guide

Welcome to the **QuickTIMS / CalTrack** repository! This is a multi-tenant workforce management SaaS platform designed to handle employee timesheets (with GPS + photo verification), scheduling, tasks, leaves, and payroll.

■ Tech Stack

- **Backend**: Python 3.13 / Django 5.2 / Django REST Framework (DRF)
- **Frontend**: React 19 / Vite / Redux Toolkit (No Tailwind, CSS variables only)
- **Database**: PostgreSQL 16 (with multi-tenant isolation via `django-tenants`)
- **Queue/Real-time**: Redis 7 / Celery / Django Channels (Daphne)

■ Prerequisites

Ensure you have the following installed on your machine: 1. **Python 3.13+** 2. **Node.js (v18+)** & npm 3. **Docker & Docker Compose**

■ Getting Started

Follow these steps to set up the project locally:

1. Database & Cache Services (Docker)

We use Docker to run PostgreSQL and Redis. 1. Navigate to the root directory. 2. Spin up the containers: `bash docker compose up -d` *Note: This starts PostgreSQL on port `5432` and Redis on port `6379`.*

---

2. Backend Setup (Django)

1. Open a terminal and navigate to the `backend` directory: `bash cd backend` 2. Create and activate a virtual environment: `bash

Windows

python -m venv .venv .venv\Scripts\activate

macOS / Linux

python3 -m venv .venv source .venv/bin/activate `3. Install the dependencies:` bash pip install -r requirements.txt `4. Copy the environment variables template and configure them:` bash cp .env.example .env `Note: Open .env and fill in DJANGO_SECRET_KEY, database credentials (if not using defaults), and Google OAuth client keys. 5. Run migrations:` bash python manage.py migrate `6. Start the development server:` bash python manage.py runserver *`Alternatively, to run with WebSocket support (Django Channels), start Daphne:`* bash daphne -b 0.0.0.0 -p 8000 quicktims.asgi:application `

---

3. Queue Services (Celery)

For handling background tasks (reports, invoice generation, email notifications), run the Celery worker and beat in separate terminals (inside the activated virtualenv in the `backend` folder):

- **Celery Worker**:

```
bash celery -A quicktims worker -l info
```

- **Celery Beat** (scheduler):

```
bash celery -A quicktims beat -l info
```

---

4. Frontend Setup (React + Vite)

1. Open a new terminal and navigate to the `frontend` directory: `bash cd frontend` 2. Install npm packages: `bash npm install` 3. Create a `.env` file for the frontend: `ini VITE_API_BASE_URL=http://localhost:8000/api VITE_GOOGLE_CLIENT_ID=your-google-oauth-client-id.apps.googleusercontent.com` 4. Run the frontend development server: `bash npm run dev` 5. Open your browser and navigate to `http://localhost:5173`.

■ Development Guidelines & Rules

- **No Tailwind CSS**: All styling is done using Vanilla CSS variables and inline styles. Brand colors can be found in `CLAUDE.md`.
- **Database Access**: Always use the Django ORM. Do not write raw queries.
- **Tenant Isolation**: Every queryset must be filtered by the request company/user. Never fetch unscoped data.
- **Named Exports**: In the frontend, always use named exports for pages and components (e.g. `export function Page()`).
- **File Uploads**: All uploads (`ImageField` / `FileField`) are capped at 5MB and validated at the model layer. Ensure you install `python-magic-bin` (on Windows) or `libmagic` (on Linux) in your environment for robust MIME checking.